

FISA DE LUCRU. PRELUCRARI ALE NUMERELOR

Disciplina: Informatica | **Clasa:** a IX-a | **Limbaj utilizat:** Python / Pseudocod

Continuturi vizate	Eficienta algoritmilor $O(n)$, operatii cu cifre, divizori si numere perfecte, CMMDC (Euclid), transformarea in baza 2.
Activitati vizate	Evaluarea complexitatii $O(n)$ fata de $O(\sqrt{n})$, determinarea CMMDC/CMMMC pentru probleme practice (semafoare, gresie), lucrul cu oglinditul unui numar.
Obiective	Elevul intelege si aplica algoritmii eficienti pentru divizori, utilizeaza algoritmul lui Euclid si converteste numere din baza 10 in baza 2.
Durata recomandata	50 minute

1. Retinem ideea principala

A. Eficienta algoritmilor (Complexitatea)

Atunci cand scriem un program, ne intereseaza sa consume cat mai putina memorie (eficienta in spatiu) si sa se execute cat mai repede (eficienta in timp). Folosim notatia Big O pentru a masura numarul de operatii. Un algoritm de complexitate $O(n)$ face un numar de operatii proportional cu n , in timp ce unul de complexitate $O(\sqrt{n})$ este mult mai rapid pentru numere mari.

B. Operatii cu cifrele unui numar

Folosim `% 10` pentru a extrage ultima cifra si `// 10` pentru a o elimina. Oglinditul unui numar se obtine construind un numar nou. Atentie: oglinditul oglinditului nu da mereu numarul initial (de exemplu, pentru 120, oglinditul este 21, iar oglinditul lui 21 este 12, zeroul s-a pierdut).

C. Divizorii unui numar si numere perfecte

Un numar natural este divizor al lui n daca n se imparte exact la el (restul este 0). Parcurgerea divizorilor pana la n este ineficienta. Este mult mai rapid sa cautam divizorii doar pana la $n/2$ sau optim pana la $\sqrt{n}$. Un numar este **perfect** daca este egal cu suma divizorilor sai strict mai mici decat el (ex: $6 = 1 + 2 + 3$).

D. CMMDC si CMMMC (Algoritmul lui Euclid)

- Cel mai mare divizor comun (CMMDC)** se foloseste practic pentru a imparti suprafete fara pierderi (ex: acoperirea unei podele cu placi de gresie patrata cat mai mari). Algoritmul lui Euclid foloseste impartiri repetate.
- Cel mai mic multiplu comun (CMMMC)** se afla folosind formula: $CMMMC = (a \times b) / CMMDC(a, b)$. Este util pentru sincronizari (ex: la ce interval de timp trec doua semafoare simultan pe culoarea verde).

E. Transformarea din baza 10 in baza 2

Orice numar (cum ar fi codul ASCII al unei litere) este stocat in calculator in baza 2 (binar). Transformarea se face prin impartiri repetate la 2 pana cand catul devine 0. Resturile obtinute, citite in ordine inversa, formeaza numarul in baza 2.

2. Exemple rezolvate

A. Eficienta determinarii divizorilor (Cautarea pana la $\sqrt{n}$).

Vrem sa afisam toti divizorii numarului 36.

- *Varianta ineficienta* $O(n)$: Verifica toate numerele de la 1 la 36. (36 de operatii)
- *Varianta eficienta* $O(\sqrt{n})$: Verifica doar numerele pana la $\sqrt{36}$ (adica pana la 6). Daca gasim un divizor d , automat cunoastem si perechea sa $n // d$. (Doar 6 operatii!)

B. Problema semafoarelor (CMMMC bazat pe CMMDC)

Semaforul A devine verde la fiecare 40 de secunde, semaforul B la fiecare 60 de secunde. Dupa cate secunde vor fi din nou verzi simultan? Trebuie sa calculam CMMMC.

```
a = 40
b = 60
produs = a * b

# Algoritmul lui Euclid prin impartiri repetate pentru CMMDC
while b != 0:
    rest = a % b
    a = b
    b = rest

cmmdc = a
cmmmc = produs // cmmdc
print("Semafoarele vor fi verzi simultan dupa:", cmmmc, "secunde.")
```

3. Exerciții diversificate

A. Completeaza spatiile libere

1. Algoritmul cel mai cunoscut pentru determinarea celui mai mare divizor comun poarta numele de algoritmul lui _____.
2. Pentru a simplifica complet o fractie data prin numarul x si numitorul y, trebuie sa impartim atat pe x cat si pe y la _____.
3. Daca analizam algoritmul de extragere a cifrelor, complexitatea acestuia depinde de numarul de _____ ale numarului procesat.
4. Pentru a converti un numar din baza 10 in baza 2, efectuam impartiri repetate la 2, iar rezultatul binar este format din _____ citite in ordine inversa.

B. Adevarat sau fals

1. Algoritmul care cauta divizorii unui numar pana la $n/2$ este la fel de eficient (rapid) ca cel care cauta divizorii pana la $\sqrt{n}$ (A / F)
2. Daca aplicam oglinditul numarului 5400, si apoi calculam oglinditul noului rezultat, vom obtine valoarea 54. (A / F)
3. Formula eficienta de calcul pentru CMMMC a doua numere necesita determinarea initiala a CMMDC-ului lor. (A / F)
4. Notatia $O(n)$ descrie complexitatea unui algoritm din punct de vedere al spatiului de memorie consumat, nu al timpului de executare. (A / F)

C. Exerciții de lucru

1. Verificarea unui Numar Perfect

Foloseste reperele de eficienta parcurgand divizorii doar pana la $n/2$. Scrie un program care citeste un numar natural n si verifica daca este numar perfect (suma divizorilor strict mai mici decat el este egala cu el insusi). Testeaza cu numarul 28.

2. Cod lacunar: Euclid cu scaderi repetate (Placi de gresie)

Avem o sala de lungime L si latime l.
Vrem sa aflam latura maxima a unei placi de gresie patrate cu care putem pava sala (CMMDC). Completeaza codul algoritmului lui Euclid bazat pe scaderi:

```
L = int(input("Lungime: "))
l = int(input("Latime: "))

while L != ____:
    if L > l:
        L = L - ____
    else:
        ____ = l - L

print("Latura maxima a placii de gresie este:", L)
```

3. Baza 10 in Baza 2

Codul ASCII al literei 'A' este 65 in baza 10. Executa pe foaie algoritmul de transformare prin impartiri repetate la 2 si scrie sirul de biti corespunzator literei 'A' (baza 2).

Barem orientativ / indicatii

- **A:** 1. Euclid; 2. CMMDC (sau cel mai mare divizor comun); 3. cifre (sau lungimea); 4. resturile.
- **B:** 1-F (Algoritmul pana la $\sqrt{n}$ face mult mai putine operatii, deci e mai rapid); 2-A (Oglinditul lui 5400 este 45, oglinditul lui 45 este 54, deci s-a pierdut valoarea originala); 3-A; 4-F (Notatia Big O se foloseste pentru a descrie in primul rand complexitatea timpului de executare, desi exista si notatii pentru memorie).
- **C1. Indicatii:** Se initializeaza suma = 0. Folosim un for i in range(1, n // 2 + 1). Daca n % i == 0, se adauga la suma. La final se compara suma == n. Pentru 28, suma este 1 + 2 + 4 + 7 + 14 = 28.
- **C2. Cod lacunar:** l (L diferit de l), l (L devine L - l), l (l devine l - L).
- **C3. Rezolvare manuala:**

65 // 2 = 32, rest 1

32 // 2 = 16, rest 0

16 // 2 = 8, rest 0

8 // 2 = 4, rest 0

4 // 2 = 2, rest 0

2 // 2 = 1, rest 0

1 // 2 = 0, rest 1

Citind resturile de jos in sus, 65 in baza 2 este **1000001**.